

Day 3 · 8/6

AI 编程能写出复杂的基础架构吗？

MQ9: Agent 通信层的实践

开源、基础架构、AI 编程 —— 三件事放在一起看

以 RobustMQ 与 MQ9 为例：能写到什么程度、边界在哪、我们是怎么做的

企业级 AI 编程实战营 · 系列直播（三）

今晚的路线

五段，从判断讲到一个真实组件



今晚的核心判断：复杂基础架构能用 AI 写，而且应该用 —— 但不能当“全自动开发”，要当“高吞吐执行器”。

重点在第 ④ 段：MQ9 是我们在做的 Agent 通信层，会讲清它的设计取舍和背后的判断。

为什么把这三件事放在一起看

开源

复杂 infra 的主要产地：Kafka、Redis、K8s 都从社区长出来

基础架构

门槛最高的一层：一个团队一年，个人过去碰不了

AI 编程

改变的不是写代码快慢，是这一层的生产方式

连起来的那句话：AI 编程把“产出”这件事的成本压下来了，于是小团队第一次真的能碰复杂 infra —— 而开源是它最自然的载体和验证场。

今晚不讲工具：讲判断、边界，和一个正在做的组件。

能用，而且应该用 —— 但要用对姿势

“基础软件当然可以用 AI 写，而且应该用；但不能把它当‘全自动开发’，而要当‘高吞吐执行器’。”

“门槛确实在降，但壁垒没有消失，它只是从编码本身，迁移到了判断力和系统能力。”

- “全自动开发”是幻想：infra 的错误路径和性能取舍，模型给不了你答案。
- “高吞吐执行器”是现实：方向和架构你定，产出交给它，速度提升是实打实的。

一句实在话

这半年的进度，没有 AI 是做不到的

“说句实在话，没有 AI，靠一个小团队的人手，这半年的进度是不可能的。”

- 存储层那一堆重构和重写 -- 背后有 AI。
- 补上的那批测试 -- 背后有 AI。
- ISR 一个个挖出来的坑 -- 背后有 AI。
- 那套能自动跑混沌演练的 harness -- 背后有 AI。

但同一篇文章里还有一句：“AI 让你快，不替你想。快是 AI 给的，稳得自己守。”

那到底哪些能交给它？

- 问题不是“AI 行不行”，而是“在 infra 这种系统里，哪一层能交给它”。
- 我们的做法是把代码分三层，每层的分工完全不同。
- 分层之后，AI 用起来就没有心理负担了 —— 你知道它错在哪也不致命。

下面几页： 三层分工 → AI 干得好的 → 不能交给它的 → 一个反向用法 → 让它当质疑者。

不能错 · 可以快 · 必须验证

① 不能错（核心层）

存储、并发控制、协议实现 —— 人主导，AI 只做辅助

② 可以快（实现层）

样板、重复改造、测试补齐、文档、重构初稿 —— AI 承担大头

③ 必须验证（收敛层）

review、测试、混沌演练 —— 人做最终验收

关键：“谁来定边界、谁来做取舍、谁来兜底线上行为，这些都不能外包。”

在 infra 项目里，这些几乎全交出去了

文档 / 博客 / 网页

对外内容几乎全部 AI 辅助完成

SDK / 集成 / 插件

多语言 SDK、生态集成这类活，量大但模式清晰

测试补齐

过去最容易拖欠的部分，现在顺手就做了

重复改造 / 重构初稿

跨模块的机械改动、结构调整

原话是这么说的：“除了 mq9 内核需要认真打磨的代码，我们几乎所有事情都会在 AI 辅助下完成。”

不能交给 AI 的部分

架构是灵魂，核心代码是命

必须自己做

- 架构与设计 —— 项目的灵魂
- 存储层、并发控制、协议实现
- 性能关键路径、疑难问题修复

AI 写出来的典型病症

- 到处 clone
- 到处 `Arc<Mutex>`
- 到处 `unwrap_or_default`

一句话点透：“AI 可以写出‘能用’的代码，但基础设施需要的是‘优雅’的代码。”为了开发速度让它写核心，会在未来付出代价。

注意：这不是能力问题，是责任问题 —— 核心代码你得能一行行解释清楚。

一个反向用法

不让他写，让它查

“我不是让 AI 帮我写代码，而是让它帮我检查代码。”

边界条件

漏掉的空值、越界、极端输入

panic 风险

哪里会炸、炸了怎么办

资源释放

连接、句柄、锁有没有正确释放

双模型互查

一个模型生成测试，另一个模型 review

为什么这招好用：核心代码的所有权还在你手上，同时又拿到了一个不知疲倦的 reviewer。

让 AI 当质疑者，逼你把话说清楚

扮演怀疑论的基础设施工程师

- 对新概念持怀疑态度、对分布式有深度理解
- 对 MQ9 提最尖锐的五问
- 换名字而已？用户存在吗？零壁垒？

四角色评审同一篇设计

- 基础架构专家 / 一线开发者
- 算法专家 / 另一个 Agent
- 各自视角挑刺，产出可执行修改建议

逼出来的结论（很值）：“MQ9 最大的风险不是技术实现，不是竞争对手，是时机。”——这种判断，靠自己很难想到。

也有被驳回的时候：“这个回答直接打破了我的预设。”

它是放大器，不是替代品

“AI 是放大器，让强者更强，也暴露弱者的不足。”

- 最大的风险是技术直觉的丢失 —— 长期不自己写核心，判断力会退化。
- 在 infra 领域，这个退化是致命的：错误路径和性能取舍全靠直觉起手。
- 所以核心层坚持自己写，不是固执，是在保养自己的判断力。

我的取舍：把 AI 用在“量”上，把自己留在“质”上。

一个真实的开源 infra 项目

定位

Rust 写的多协议统一消息引擎：单二进制、零外部依赖、底层一份统一存储

原生协议

MQTT / Kafka / AMQP / NATS, MQ9 是第五个协议层

当前进度

0.4.0 四协议链路打通、MQ9 第一版内核诞生；现已到 0.4.7

下半年

MQ9 加语义拦截与审计、支持 Kafka 协议、存储探索 S3

整条链路的野心：物联网设备 → 大数据平台 → AI Agent, 用一份存储、多协议视图打通。

不发明新协议

- MQTT、Kafka、NATS、AMQP 这些协议，早已内化在大模型的训练语料里 —— 采用成本几乎为零。
- 发明一套新抽象是危险的：没人懂、没生态、AI 也不会写，等于自己给自己加门槛。
- 所以 MQ9 建在 NATS 协议之上，不造新线上协议指令 —— 任何 NATS 客户端都能直连。

文本协议

AI 原生友好，模型天然会写

Subject 层级路由

天然适合邮箱式寻址

生态现成

官方 9 种语言 SDK，社区 30+

这条取舍的本质：在 AI 时代，“模型会不会写你的协议”本身就是一种生态优势。

统一存储 + 多协议层，存储分三级

【图片占位】

建议贴：RobustMQ 架构图（多协议接入层 → Broker → 统一存储；Meta Service / SearchEngine）

Memory

realtime: 最快，不落盘

RocksDB

reliable: 可靠，本地持久化

File Segment

replay: 可回放，适合大容量

设计原则：store-first delivery —— 先写存储再推送，持久化是默认而不是可选项。

最能说明问题的一块

AI 驱动的混沌测试体系

- 闭环：自动部署集群 → 注入故障 → 跑多语言 SDK 测试 → 分析日志 → 生成报告。
- “没有 hardcode 的执行流程，没有 Python 调度逻辑，AI 驱动一切”——而且这套系统本身完全由 Claude Code 实现。

26 个场景

MQTT 13 + MQ9 13，分基础/故障/客户端异常/组合

跑的频率

P0 每 2 小时 · P1 每天凌晨 · P2 每周

工期压缩

首期从预估 4-6 周压到 1-2 周

但真正难的地方：“场景库是持续输入的地方……这件事比写代码难。”

先看一个真实场景

- 一个研究 Agent 接到任务，把它拆成子任务，派给多个搜索 Agent。
- 整个过程跨 3 小时：中途还会派生新任务、有的失败要重试、有的先回传中间结果。
- 主 Agent 大部分时间在做一件事 —— 等待。

于是问题来了：这种“跨小时、会失败、要重试、多对多”的通信，用什么承载？HTTP 撑不住，Kafka 又不是为这个设计的。

注意：这不是吞吐问题 —— 每个 Agent 的流量都很小，难的是“状态和身份”。

四条路都差一点

HTTP 同步调用

要求双方同时在线；等几小时必然超时；故障会沿链路传导

SSE

断线就丢，没有补投机制

Webhook

fire-and-forget，没有投递保证

Kafka / NATS 裸用

要自己拼 topic + consumer group；核心抽象不是“Agent 身份”

共同的缺口：没有一层东西，天生就把“Agent 的身份 + 离线可达 + 可靠投递”当成默认。

AI Agent 的通信层，核心抽象是 mailbox

“MQ9 是 AI Agent 的 communication layer，核心抽象是 mailbox。”

MQ9 Broker

协议无关的可靠消息管道：持久化 mailbox、注册中心、N-to-N 拓扑

MQ9 SDK

协议的轻量封装层：支持 A2A 等，也支持自定义协议

一句类比：mem0 填的是记忆层，MQ9 填的是消息层 —— Agent 协作里默认就该有的那一层。

管道 vs 收件箱：心智模型不同

传统 MQ = 管道

- topic / subject 是无名的、中性的
- 核心指标是吞吐与顺序
- “发到 topic，谁拿到算谁的”
- 假设消费者持续在线

MQ9 = 收件箱

- mailbox 属于某一个 Agent
- mail_address 就是身份
- “A 发给 B”，离线投递是默认
- 核心是身份与状态，不是吞吐

出发点其实很朴素：Agent 之间通信应该像发邮件一样简单 —— 邮件系统跑了几十年，靠的正是“收件箱”这个抽象。

subject pattern 即 API

- 不引入新的线上协议指令：全部用 NATS 的 PUB/SUB + JSON 承载（借鉴 JetStream 的 \$JS.API.* 思路）。
- 寻址即 subject: \$mq9.AI.MAILBOX.CREATE、\$mq9.AI.MAILBOX.{id}、\$mq9.AI.PUBLIC.LIST。

信箱与消息

MAILBOX.CREATE
MSG.SEND / FETCH / ACK
MSG.QUERY / DELETE

注册与发现

AGENT.REGISTER / UNREGISTER
AGENT.DISCOVER
AGENT.REPORT

好处：任何 NATS 客户端（8 种语言）都能直连，认证也复用 NATS 的那套（token / JWT / nkey）。

哪些是刻意不做的

三级优先级

critical / urgent / normal; 优先级内严格有序, 跨级不保证

幂等是能力不是承诺

提供 dedup_id, 但 exactly-once 本质不存在

两层 TTL

mailbox 级 TTL + 消息级 expire_at

消息不分片

超过上限 (如 10MB) 直接拒绝, 不做分片

mail_address 即凭证

多租户在连接层实现, 协议层零感知

取舍的逻辑: 简单能跑很久, 复杂会先把自己压垮。

Agent 找到 Agent，是这一层的地基

- AGENT.REGISTER 注册身份与能力，AGENT.DISCOVER 按需查找。
- 四种检索方式：按 key、按 tag、全文检索、语义（向量）检索。

SearchEngine

RobustMQ 新增模块，为发现能力打底

LanceDB

纯 Rust 嵌入式，向量 + 全文（Tantivy BM25）

fastembed-rs

默认 BGE-small: 130MB、384 维

异步索引

索引失败不影响主写入链路

轻 · 稳 · 开

轻

A2A Agent 从 60 行变 15 行，半小时闭环

稳

不丢消息，也不把业务拖下水

开

与 a2a-sdk 100% 兼容，不绑协议，可与 Kafka / RabbitMQ 共存

边界也划得很清楚：只进四个真空区 —— Agent 发现、可靠异步传输、监控追踪、协议中立的基础设施；不做协议本身、不做编排、不做框架、不做全家桶。

变了的那条假设是“端点”

旧假设：端点是设备

- 哑的 -- 只收发字节
- 廉价的 -- 坏了换一个
- 稳定的 · 确定的
- 难点在数据面（吞吐）

新假设：端点是智能体

- 聪明的 -- 会自己决策
- 昂贵的 -- 每次调用都花钱
- 易逝的 · 不确定的
- 难点在控制面（秩序）

由此推出的两条：① 瓶颈从“单通道搬多少数据”，挪到“管理海量端点的存在、状态和位置”——百万级端点、每个流量都很小；② 发现、可靠、追踪、管控这些职责，会从应用层被逼回基础设施。

MQ9 要做的四件事

前两件是地基，后两件才真正难

discovery

Agent 找到 Agent

地基

durable mailbox

离线也能收到

地基

correlation

一件事的全链路追踪

难

identity + 拦截

身份与管控

难

后两件难在哪：要求 broker 理解流过的不是字节，而是“一个 Agent 正在做的、有状态、需要被约束的一件事”。

诚实地说：“从端点变了到这些推论，每一步都还是假设，不是结论。”这条线还很早，我们赌的是方向。

现在做到哪

不夸大，如实说

已有的

MQ9 第一版内核、Agent 注册与发现、a2a over mq9 探索、语义检索打底

能用起来的

NATS 客户端 8 语言直连、langchain-mq9 集成、demo server

还在路上

MCP Server 仍是技术方案、SDK 尚在计划中（约 2000 行 Python，4-6 周）

下半年

语义拦截与审计、与 a2a 更紧结合、Kafka 协议、S3 存储

一句话：broker 是主体，也是护城河（多年积累）；SDK 是最后一环。

回到开源、infra、AI 编程这三件事

AI 编程

把“产出”的成本压下来



小团队

第一次能碰复杂 infra



开源

最自然的载体与验证场

“AI Coding 时代，人多不一定是优势。多一个人多一份沟通摩擦，多一份协调成本。”

那稀缺的是什么：“真正稀缺的不是技能，是那股想搞清楚、想做出来的劲儿。”

四条现实路径

① 先读懂一个

把一个成熟组件的核心链路读透

② 从场景库入手

写混沌测试场景 -- 门槛低但价值高，而且“比写代码难”

③ 接生态的活

SDK、集成、插件 -- 模式清晰，最容易出成果

④ 再碰内核

有了上下文，再去动存储、协议这些硬骨头

顺序建议：跳过第①步直接写内核，通常做出来的是玩具。

五句话

- 复杂基础架构能用 AI 写，而且应该用 —— 但当它是“高吞吐执行器”，不是“全自动开发”。
- 分三层：核心层人主导、实现层 AI 承担大头、收敛层人做最终验收。
- 一个反向用法很值：不让它写核心，让它查边界条件、当质疑者、做多角色评审。
- MQ9 的关键不是又一个 MQ，而是把心智模型从“管道”换成“收件箱”——身份和状态优先于吞吐。
- 变了的那条假设是端点：从哑的设备变成聪明、昂贵、易逝的智能体，于是职责被逼回基础设施。

三天串起来

同一套方法，三个层次

Day 1 · Agent 层

企业底座怎么设计，AI 怎么帮你实现

Day 2 · 组件层

Loop Engineering 的实践，与对 OPC 的提效

Day 3 · 基础架构层

用 AI 做复杂 infra，以 MQ9 为例

- 2 期班 9 月初开班：11 周、四个真实开源项目，产出是公开的贡献记录。
- 详细课表与报名方式，看群内公告或问助教。

谢谢

AI 让你快，不替你想 —— 快是 AI 给的，稳得自己守

RobustMQ / MQ9 欢迎参与：认领 issue · 写场景 · 提 PR

【占位：仓库地址 / 二维码 / 报名信息】